

Introductory Econometrics with Recitation: TA Session Week 3

Danny Po-Hsien Kang (康柏賢)

March 10, 2026

Today's agenda

- 1 Control Flow
 - for-loop
 - if-else
 - function()
- 2 Row / Column Sums
 - rowSums()
 - colSums()
- 3 Hypothesis Testing: t-test
 - One-Sample t-test
 - Paired Sample t-test
 - Two Independent Sample t-test

Today's Dataset

- Please download the three datasets from the OpenIntro website.
 - `china.csv`: Weekly time spent on child care in China.
 - `textbooks.csv`: Prices of textbooks for various courses at UCLA Bookstore and Amazon.com.
 - `ncbirths.csv`: A 2004 dataset from North Carolina on births, including mothers' habits and practices, with 1,000 cases.
- After that, import the data

```
china <- read.csv("data/china.csv")  
book <- read.csv("data/textbooks.csv")  
birth <- read.csv("data/ncbirths.csv")
```

Repeated Work: for-loop

- **Syntax:**

```
for (var in seq){  
    # do something  
}
```

- **Example:**

```
for (i in 1:5){  
    print(i)  
}
```

- **Output:**

```
[1] 1  
[1] 2  
[1] 3  
[1] 4  
[1] 5
```

Different Works in Different Conditions: if-else

- **Syntax:**

```
if(a < b){  
    # do work 1  
}else if(a == b){  
    # do work 2  
}else{  
    # do work 3  
}
```

Different Works in Different Conditions: if-else

- **Example:**

```
for (i in 55:65){  
  if(i < 60){  
    print("Fail.")  
  }else if(i == 60){  
    print("On the threshold.")  
  }else{  
    print("Pass the course!")  
  }  
}
```

Different Works in Different Conditions: if-else

- **Output:**

```
[1] "Fail."  
[1] "Fail."  
[1] "Fail."  
[1] "Fail."  
[1] "Fail."  
[1] "On the threshold."  
[1] "Pass the course!"  
[1] "Pass the course!"  
[1] "Pass the course!"  
[1] "Pass the course!"  
[1] "Pass the course!"
```

Self-Defined Function: function()

- **Syntax:**

```
function_name <- function(param1, param2, ...){  
  # do something  
  return(output)  
}
```

- **Example:**

```
add <- function(a, b){  
  return(a+b)  
}
```

- **Output:**

```
print(add(50, 100)) # or use add(a = 50, b = 100)  
[1] 150
```

Reasons to use function()

- Avoid copying and pasting the same code when repeated use.
 - Makes code cleaner.
- When you hope the parameter can be customized.
 - Sometimes the parameters in a project are undetermined.
 - Without a function, changing parameters can be troublesome and easy to make mistakes.
- Functions can be saved in a script.
 - Can be called by `source("myfunctions.R")`.
 - This allowed the functions to be portable.
 - Makes the code shorter and easier to understand.

Row-wise Totals in R: rowSums()

- **Syntax:**

```
df %>% rowSums()
```

- **Example:**

```
df <- data.frame(  
  a = c(1, 2, 3),  
  b = c(4, 5, 6)  
)  
  
# Add new column: Method 1  
df$row_total1 <- df %>% rowSums()  
  
# Add new column: Method 2  
df["row_total2"] <- df %>% rowSums()  
  
print(df)
```

- **Be aware of non-numeric columns in data.**

Row-wise Totals in R: rowSums()

- **Output:**

	a	b	row_total1	row_total2
1	1	4	5	10
2	2	5	7	14
3	3	6	9	18

- Be aware of non-numeric columns in data.

Column-wise Totals in R: colSums()

- **Syntax:**

```
df %>% colSums()
```

- **Example:**

```
df <- data.frame(  
  a = c(1, 2, 3),  
  b = c(4, 5, 6)  
)  
  
col_total <- df %>% colSums()  
print(col_total)  
  
# Output: named vector  
# a  b  
# 6 15
```

One-Sample t-test: `t.test()`

- **When to use:** Test whether the population mean equals a target μ .
- **Syntax:**

```
t.test(vec, mu,  
       alternative, conf.level)
```

- **Example 1:**

```
china <- china %>% filter(child_care >= 0)  
  
china_male <- china %>% filter(gender == 1)  
t.test(china_male$child_care,  
       mu = 25,  
       conf.level = 0.95)
```

- Default value for parameter `alternative` is `"two.sided"`
 - Use `"less"` or `"greater"` for one-sided tests.
 - The results of `t.test()` will be saved in a list. You can use `$` or `[[ ]]` to extract specific values (e.g. p-value, confidence interval).

One-Sample t-test: `t.test()`

- **Output:**

```
One Sample t-test

data:  china_male$child_care
t = -9.2583, df = 357, p-value < 2.2e-16

alternative hypothesis: true mean is not equal to 25

95 percent confidence interval:
 13.17385 17.31777

sample estimates:
mean of x
15.24581
```

One-Sample t-test: `t.test()`

- **Example 2:**

```
t.test(china$child_care,  
      mu = 25,  
      alternative = "less",  
      conf.level = 0.95)
```

One-Sample t-test: `t.test()`

- **Output:**

One Sample `t`-test

```
data: china$child_care
```

```
t = -0.73041, df = 1021, p-value = 0.2327
```

```
alternative hypothesis: true mean is less than 25
```

```
95 percent confidence interval:
```

```
-Inf 25.79633
```

```
sample estimates:
```

```
mean of x
```

```
24.36497
```

Paired Sample t-test: `t.test()`

- **When to use:**

- Pre/post on the same subjects or two methods measured on the same items.
- You want to test whether the mean difference equals μ (usually 0).

- **Syntax:**

```
t.test(vec_1, vec_2,  
       paired = TRUE, conf.level)
```

- **Example 1:**

```
t.test(book$ucla_new,  
       book$amaz_new,  
       paired = TRUE,  
       coef.level = 0.95)
```

Paired Sample t-test: `t.test()`

- **Output:**

Paired t-test

data: book\$ucla_new and book\$amaz_new

t = 7.6488, df = 72, p-value = 6.928e-11

alternative hypothesis: true mean difference is not equal to 0

95 percent confidence interval:

9.435636 16.087652

sample estimates:

mean difference

12.76164

Two Independent Sample t-test: `t.test()`

- **When to use:**

- Comparing the means of two distinct groups (e.g., treatment vs control; smokers vs non-smokers).

- **Syntax:**

```
t.test(data$ValueVar ~ data$GroupVar,  
       conf.level)
```

- **Example 1:**

```
t.test(birth$weight ~ birth$habit,  
       conf.level = 0.95)
```

Two Independent Sample t-test: `t.test()`

- **Output:**

Welch Two Sample t-test

```
data:  birth$weight by birth$habit
```

```
t = 2.359, df = 171.32, p-value = 0.01945
```

```
alternative hypothesis: true difference in means between  
group nonsmoker and group smoker is not equal to 0
```

```
95 percent confidence interval:
```

```
0.05151165 0.57957328
```

```
sample estimates:
```

```
mean in group nonsmoker  
7.144273
```

```
mean in group smoker  
6.828730
```